

CSE207 Practice Questions

1. Suppose you are given a string S and you have to find the length of the string. Show the recursive definition of the problem by writing only the recursive function.

Example: Input string: one two three four five

Output: Length = 23

2. Ackermann's number can be calculated using the recursive definition below. Write a recursive algorithm that calculates Ackermann's number. Then manually verify the algorithm for all of the following original test cases: Ackermann(2, 3), Ackermann(3, 0), Ackermann(2, 5), and Ackermann(0, 3).

$$\begin{aligned} A(m, n) &= n + 1 && \text{if } m = 0 \\ &A(m - 1, 1) && \text{if } n = 0 \text{ and } m > 0 \\ &A(m - 1, A(m, n - 1)) && \text{otherwise} \end{aligned}$$

3. Write only a recursive function that counts the number of digits in a positive integer. For example, input 50724 should return 5.

4. Write a recursive function to find the sum of the elements of an integer array. The function must not use any loop.

5. Find the minimum and maximum number of nodes of a binary tree of height 6.

6. A binary tree has eight nodes. The postorder and inorder traversals are given below. Draw the binary tree.

Postorder: F E C H G D B A

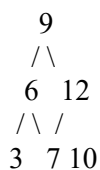
Inorder: F C E A B H D G

7. Write an algorithm that determines whether a binary tree is complete.

8. List the nodes of the following tree in preorder and postorder.



9. Given a binary tree, write an algorithm to find the sum of all left leaves. For the tree below, the sum of all left leaves is $10 + 3 = 13$.



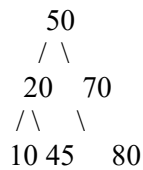
10. Write a recursive function to count the total number of leaf nodes in a binary tree.

11. Given preorder and inorder traversals of a binary tree, reconstruct and draw the tree. Use: Preorder = A B D E C F and Inorder = D B E A C F.

12. Write a function to determine the height of a binary tree. Your solution must be recursive.

13. Write a program to convert an array into a Binary Search Tree by inserting the elements as nodes in the given array order. Also check whether the created BST is a near-complete tree or not.

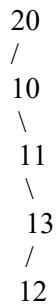
14. Write only a function that displays every node in a BST that has at least one child. For the following BST, the output should be 50, 20 and 70.



15. Given the preorder traversal of a BST, write a function to check whether each non-leaf node has only one child. Assume that the BST contains unique entries.

Example: pre = {20, 10, 11, 13, 12}

Output: Yes



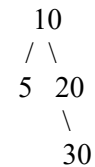
16. Given a Binary Search Tree, write a function to find the second largest element.

Example 1:



Output: 5

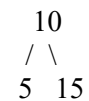
Example 2:



Output: 20

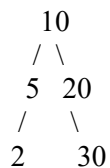
17. Given a Binary Search Tree, write a function to find the second smallest element.

Example 1:



Output: 10

Example 2:



Output: 5

18. Given a BST, write a function to find the third largest element without storing the complete inorder traversal in an array.

19. Write a function to check whether a given binary tree satisfies the Binary Search Tree property.

20. Given a BST and a key value, write a function to print the inorder predecessor and inorder successor of that key, if they exist.

21. Construct an AVL tree by inserting the following values sequentially. Show the balance factor of each node in the resulting tree. Then delete 56 and 14, one at a time. Draw the tree after each deletion and apply the required rotations so that the tree remains balanced.

80 70 66 56 33 23 14 10 7

22. Construct an AVL tree by inserting the values 30, 20, 10, 25, 28, 27 in order. Show every rotation performed and the final balance factors.

23. Starting from an empty AVL tree, insert 50, 30, 70, 20, 40, 60, 80, 10 and then delete 80 and 70. Draw the tree after each deletion and rebalance it where necessary.

24. Write a program to create an undirected graph using an adjacency matrix and display it.

Input: Number of vertices n, number of edges e, followed by e pairs of vertices.

Output: Adjacency matrix

Data Structure: Array

Sample Input:

4
4
0 1
0 2
1 2
2 3

Sample Output:

0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0

25. Write a program to create an undirected graph using an adjacency list and display it.

Data Structure: Linked List

Sample Input:

4
4
0 1
0 2
1 2
2 3

Sample Output:

0 -> 1 -> 2
1 -> 0 -> 2
2 -> 0 -> 1 -> 3
3 -> 2

26. Write a function to add an edge between two vertices in an existing undirected graph represented using an adjacency list.

Data Structure: Linked List

Sample Input: Edge to add: 2 3

Sample Output:

2 -> 1 -> 3

3 -> 2

27. Write a function to remove an edge between two vertices in an undirected graph represented using an adjacency list.

Data Structure: Linked List

Sample Input:

4

4

0 1

0 2

1 2

2 3

Edge to delete: 1 2

Sample Output:

1 -> 0

2 -> 0 -> 3

28. Write a function to check whether two vertices are directly connected.

Data Structure: Array

Sample Input:

4

3

0 1

1 2

2 3

Check: 1 2

Sample Output: Adjacent

29. Write a function to calculate the degree of each vertex in an undirected graph.

Data Structure: Array

Sample Input:

5

4

0 1

0 2

1 3

1 4

Sample Output:

0 -> 2

1 -> 3

2 -> 1

3 -> 1

4 -> 1

30. For a directed graph, write a function to calculate the indegree and outdegree of every vertex.

Data Structure: Array

Sample Input:

```
4
5
0 1
0 2
1 2
2 3
3 1
```

Sample Output:

```
0 -> In: 0 Out: 2
1 -> In: 2 Out: 1
2 -> In: 2 Out: 1
3 -> In: 1 Out: 1
```

31. Write a function to display all neighbors of a given vertex.

Data Structure: Linked List

Sample Input:

```
4
3
0 1
1 3
0 2
Vertex: 1
```

Sample Output: Neighbors of 1: 0 3

32. Write a function to search for a vertex and report whether it exists in the graph.

Data Structure: Linked List

Sample Input:

```
5
10 20 30 40 50
Search: 30
```

Sample Output: Found

33. Write a function to count the total number of edges in an undirected graph represented using an adjacency list.

Data Structure: Linked List

Sample Input:

```
4
4
0 1
0 2
1 2
2 3
```

Sample Output: Total edges = 4

34. Write a function to insert a new vertex into an undirected graph and connect it with the given vertices.

Data Structure: Linked List

Sample Input:

```
3
2
0 1
```

1 2
New vertex: 3
Connections: 1 2

Sample Output:

0 -> 1
1 -> 0 -> 2 -> 3
2 -> 1 -> 3
3 -> 1 -> 2

35. Write a function to delete a vertex and remove all of its incident edges.

Data Structure: Linked List

Sample Input:

4
4
0 1
0 2
1 3
2 3
Delete: 1

Sample Output:

0 -> 2
2 -> 0 -> 3
3 -> 2

36. Given an undirected graph represented by an adjacency matrix, write a function to count the total number of edges without using the original edge list.

37. Write a function to find and display all isolated vertices (vertices with degree 0) in an undirected graph.

38. Write a function to convert an undirected graph from adjacency-matrix representation to adjacency-list representation.

39. Write a function to check whether an undirected graph represented by an adjacency matrix contains any self-loop.

40. Write a function to delete a vertex from a graph represented by an adjacency matrix. Display the updated matrix after deletion.

41. Construct a Max Heap by inserting the following elements sequentially. Show the heap after each insertion.
25, 40, 15, 50, 10, 35, 60, 20

42. Given the following Max Heap, delete the root node and reconstruct the heap. Show all necessary steps.

```
90
 / \
70  80
 / \  / \
40 60 50 30
```

43. Write a function to insert a new element into a Min Heap while maintaining the heap property.

Example:

Input Heap: {10, 20, 15, 30, 40, 25}

New element: 5

Output: Display the resulting Min Heap.

44. Write an algorithm to convert an unsorted array into a Max Heap using the heapify operation. Apply your algorithm to:

{12, 7, 25, 15, 28, 33, 18, 10}

Show the resulting Max Heap.

45. Apply Heap Sort to arrange the following elements in ascending order. Show the heap after each deletion of the root.

45, 20, 35, 10, 50, 30, 25